

Technical Interview Guide

A technical-lead preparation pack for a 40 LPA engineering bar

How to use this guide

Practice the 90-second pitch, select three workflows, draw the architecture, and rehearse the question bank aloud. Use evidence from the repository and state production limitations before the interviewer discovers them.

Inside

Hiring rubric / architecture / security / AI and agentic AI / data correctness / reliability / 45 interview questions / system-design drills / live demo script

Prepared from the implemented repository - August 2026

Verify test counts, live integrations, and deployment status again before an interview.

INTERVIEW PURPOSE

PURPOSE: Explain Vidhya Vedha as an engineering system with clear ownership, trade-offs, evidence, and limitations. Do not present it as a list of screens or as nine live government integrations.

1. What a 40 LPA technical interviewer expects

At this level, I am not hiring someone because they used React, MongoDB, or an LLM. I am looking for evidence that the candidate can take an ambiguous problem, define safe boundaries, design the system, make trade-offs, ship it, operate it, and explain what would break next.

Dimension	Weight	What strong evidence looks like
Problem framing and scope	10%	Defines the resident problem, user, non-goals, and success measures before discussing features.
Architecture and system design	20%	Explains boundaries, request/data flows, consistency choices, scale limits, and alternatives.
Backend and data correctness	15%	Shows ownership checks, indexes, state transitions, conflict handling, retention, and failure semantics.
Security and privacy	15%	Threat-model-driven controls, least privilege, consent, secrets, CSRF/session reasoning, and residual risk.
AI and agentic judgment	15%	Grounding, structured output, deterministic fallbacks, approval gates, evaluation, cost, and safe autonomy.
Reliability and delivery	15%	CI, deployment, health/readiness, smoke tests, observability, rollback, and operational limitations.
Ownership and communication	10%	Uses "I designed/implemented/changed" precisely, quantifies evidence, and answers trade-offs without bluffing.

Signals that make me lean hire

- You start with the user problem and the trust boundary, not the tech stack.
- You select two or three deep workflows instead of reciting every feature.
- You can trace one request from UI to database and back, including failure paths.
- You explain why the database, not application memory, protects booking consistency.
- You distinguish AI suggestions from deterministic policy and from irreversible actions.
- You volunteer limitations before I discover them.
- You use production evidence: green CI, live health checks, deployed smoke tests, security controls, and rollback thinking.
- You can redesign the system for 10x and 100x scale without pretending the current version already supports it.

Signals that make me reject

- "It has nine services" is the entire value proposition.
- Calling official website links "integrations" or claiming government submissions/status when none exist.
- Saying the AI "knows eligibility" or the agent "automatically does everything."
- Describing authentication as secure only because passwords are hashed.
- Treating a frontend availability check as protection against double booking.
- Claiming production readiness while ignoring process-local rate limiting, an in-process reminder worker, local vault storage, monitoring, backup, and provider onboarding.
- Quoting team-level work as personal ownership without explaining your exact contribution.

2. The answer structure to use in the interview

Use this sequence every time:

1. Problem and user.
2. Product boundary and what the system does not claim.
3. Three workflows that prove engineering depth.
4. Architecture and data flow.
5. The hardest decisions and trade-offs.
6. Security, privacy, and AI safety.
7. Testing, deployment, and production evidence.
8. Current limitations and the next design.
9. Measurable outcome or the metrics you would instrument.

3. One-sentence and 90-second explanation

One sentence

Vidhya Vedha is a deployed civic-service companion for rural Indian residents that turns confusing service discovery into guided, owner-scoped workflows for preparation, booking, reminders, status capability, and human handoff, while keeping official submissions and sensitive authority data with the responsible provider.

90-second interview pitch

"I built Vidhya Vedha to solve a gap I saw in rural digital-service access: people can find links, but they often do not know which service applies, what to prepare, where responsibility changes, or how to recover when they get blocked.

I redesigned the project from a directory of forms into three deep workflows. The first is a resident companion: structured intake ranks reviewed service journeys, optionally uses OpenAI only to refine approved candidates, creates a document-readiness checklist, generates a previewable draft/PDF, and can schedule consented reminders or a human handoff. The second is healthcare scheduling, where database-level partial unique indexes prevent concurrent double booking and owner checks protect the appointment lifecycle. The third is roadside dispatch, which uses role-separated queues and an explicit state machine.

Technically, it is a React/Vite SPA with an Express API deployed as a Vercel serverless function and MongoDB Atlas for persistent state. Authentication uses opaque HTTP-only cookie sessions, server-side token hashes, per-session CSRF tokens, idle and absolute expiry, session revocation, Google ID-token verification, RBAC, Zod validation, and rate limits.

For AI safety, the model never chooses arbitrary services or directly mutates records. It receives reviewed context, uses structured output, falls back deterministically, and can only propose one allowlisted plan-task update that expires and requires a second explicit approval request.

The current release has CI, a production deployment, protected main, repository hygiene, and a scheduled 13-check production smoke test. I explicitly label provider and authority integrations as fixtures or not connected. The next production steps are shared rate limiting, an external job queue, managed encrypted object storage/KMS, centralized observability, and backup-restore evidence."

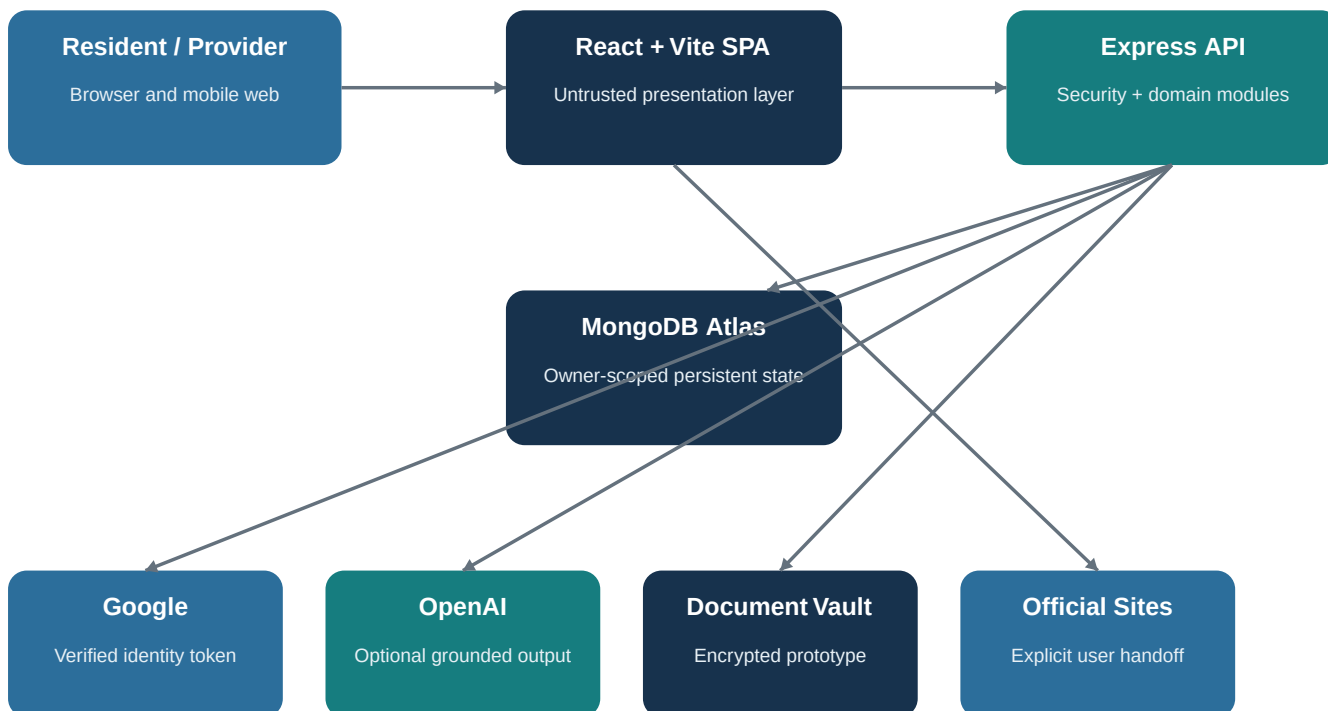
What this pitch proves

- Product judgment: the problem is not "missing websites" but decision, preparation, and recovery friction.
- Architecture: a clear browser/API/database boundary.
- Correctness: database constraints and state machines.
- Security: sessions, CSRF, RBAC, validation, and privacy minimization.
- AI maturity: bounded generation, fallback, approval, and no false autonomy.
- Production honesty: evidence plus explicit remaining work.

4. Five-minute explanation map

Time	Topic	What to say
0:00-0:40	Problem	Residents face fragmented services, uncertain preparation, language/access barriers, and unclear escalation.
0:40-1:15	Boundary	Vidhya Vedha guides and manages internal preparation; official authorities still own eligibility, submission, payment, and official status.
1:15-2:20	Three workflows	Companion, healthcare booking, roadside dispatch. Explain one hard engineering problem in each.
2:20-3:10	Architecture	React/Vite -> same-origin Express API -> MongoDB Atlas; optional Google/OpenAI; official links are explicit handoffs.
3:10-4:05	Security and AI	Opaque sessions, CSRF, owner scoping, RBAC, structured model output, deterministic fallback, approval-gated action.
4:05-4:40	Delivery evidence	Tests, build, CI, Vercel, readiness endpoint, production smoke, branch protection, dependency/secret hygiene.
4:40-5:00	Limitations	Shared limiter, queue/worker, managed vault, observability/backups, provider contracts and official connectors.

5. Architecture you should draw



Solid arrows show request or verified integration boundaries. Official authority pages remain a browser handoff unless a registered adapter exists.

Layer responsibilities

Layer	Responsibility	Interview detail
React + Vite SPA	Guided journeys, consent, previews, role-aware navigation, accessibility, CSRF-aware API client	The browser is untrusted; it never decides authorization or booking validity.
Express middleware	Helmet, CORS allowlist, body limits, sanitization, session lookup, CSRF, rate limits, Zod validation, authentication and roles	Middleware ordering matters: parse/limit -> sanitize -> session -> CSRF -> limiter -> route validation/authorization.
Controllers	Translate validated HTTP requests into domain operations and stable status/error responses	Controllers should remain thin; domain invariants belong in services/models.
Domain services	Recommendations, AI grounding, scheduling, state transitions, reminders, connectors, encryption, PDF generation, redaction	This is where business rules are testable without HTTP.
Mongoose models	Ownership references, enums, TTLs, unique/partial indexes, optimistic concurrency	Database constraints protect invariants under concurrent requests.
MongoDB Atlas	Users, sessions, assessments, plans, appointments, actions, reminders, trackers and metadata	Serverless connection reuse avoids reconnecting for every warm invocation.

External boundaries

Google ID verification, optional OpenAI,
official website handoffOnly Google/OpenAI are configured
external integrations; most authority/
provider APIs are not connected.

Request flow to narrate

1. React sends an HTTPS request to /api with the secure session cookie.
2. For authenticated mutations, Axios also sends the in-memory X-CSRF-Token.
3. Express applies security headers, exact CORS rules, request-size limits, payload sanitization, session lookup, CSRF, rate limits, validation, authentication, and role/ownership checks.
4. The controller calls a domain service.
5. The service checks current state and performs an owner-scoped MongoDB operation.
6. Database indexes or atomic updates protect concurrency-sensitive invariants.
7. The API returns an explicit success/error code; the UI refreshes authoritative state.
8. Optional OpenAI/Google calls cross a separate trust boundary and are verified before local state changes.

Why a modular monolith was reasonable

For the current traffic and team size, a modular monolith keeps transactions, ownership rules, testing, deployment, and debugging simple. The domains are separated in routes/controllers/services/models, so the code can be extracted later. Starting with microservices would add network failure, distributed tracing, deployment, schema ownership, and consistency cost before there is evidence that independent scaling is needed.

A strong answer also names extraction triggers: reminder throughput needs a queue, status connectors need independent failure isolation, vault processing needs secure object storage workers, or provider operations require tenant-specific scale and deployment ownership.

6. The three workflows to explain deeply

Workflow A: Resident service companion

Problem: residents often know the outcome they need but not the department, service name, preparation sequence, or escalation route.

- Flow:**
1. A signed-in resident enters a goal, domain, life stage, urgency, language, and optional district/description.
 2. A deterministic scorer ranks only nine reviewed journeys using goal/domain/life-stage matches and bounded keywords.
 3. If an OpenAI key exists, strict structured output may reorder and explain only those approved candidates.
 4. Returned service codes are checked against the server-owned catalogue; any exception falls back to deterministic rules.
 5. The owner can create a readiness checklist, draft/PDF, reminder, status tracker, or human handoff.
 6. The system never declares legal/medical/financial eligibility or submits an official application.

Hard engineering points:

- Safe degradation: the complete workflow works without an LLM.
- Privacy minimization: no Aadhaar, bank account, OTP, medical record, or document field exists in the intake schema.
- Owner scoping: assessments and downstream records are queried by both public ID and authenticated user ID.
- Explainability: each result contains a reason, bounded confidence, next steps, authority boundary, and official handoff.
- Scope control: the model cannot invent a tenth service.

Likely interviewer challenge: "This is just a rules engine. Why call it AI?"

Strong response: "The deterministic engine is intentionally the source of availability and safety. The AI contribution is constrained language understanding, re-ranking, multilingual explanation, and grounded conversation. I do not use AI where deterministic policy is more reliable. The product is stronger because it degrades safely rather than becoming unavailable without a model."

Workflow B: Healthcare scheduling

Problem: showing available slots is easy; preserving availability under concurrent confirmation is the real problem.

- Flow:**
1. Public provider and availability APIs generate candidate slots from reviewed weekly schedules in Asia/Kolkata.
 2. The API rejects invalid times and requires a minimum lead time.
 3. On confirmation, an owner-scoped appointment is created.
 4. A partial unique index on (providerCode, startTime) where status is booked prevents two active bookings for the same slot.
 5. A second partial unique index on (userId, startTime) prevents one resident holding overlapping active bookings at the same time.
 6. Duplicate-key races map to 409 SLOT_UNAVAILABLE; the UI refreshes availability.
 7. Cancellation/rescheduling checks ownership, current status, and the two-hour modification window.

Why this answer is senior:

- Frontend disabling is user experience, not concurrency control.

- A pre-insert "is slot free?" query is still race-prone.
- The database is the serialization point for the invariant.
- A 409 is a domain conflict, not a generic 500.
- Cancellation frees the partial-index constraint because cancelled records are excluded.

What is not production-ready: providers are seeded fixtures; there is no clinic onboarding, leave/holiday calendar, payment, medical record, provider tenancy, audit export, or clinic-system integration.

Workflow C: Roadside emergency dispatch

Problem: a request crosses resident and dispatcher roles and must move only through allowed states without pretending to replace emergency services.

- Flow:**
1. The resident selects a bounded assistance type and provides minimum contact/location context.
 2. The request enters the dispatcher queue in requested state.
 3. A dispatcher/admin can assign it and advance it through the allowed state machine.
 4. The resident sees only the owned request and can cancel only in permitted states.
 5. Provider operations returns redacted queue projections rather than raw personal records.

Hard engineering points:

- Role separation between residents and dispatchers.
- Explicit state transitions rather than arbitrary status strings.
- Minimum data collection for a high-risk workflow.
- Redacted operational views.
- Product copy clearly states that Vidhya Vedha does not contact 112 or guarantee response time.

Why these three, not all nine

These workflows demonstrate different engineering depth: decision support and safe AI, concurrency and lifecycle correctness, and role-separated operations/state machines. Education, finance, farming, utilities, ecommerce, and home service demonstrate reusable patterns, but presenting all nine equally makes the project sound like feature dumping.

7. AI and agentic AI - the explanation expected at senior level

What is actually AI

Capability	AI role	Deterministic control
Companion selection	Optional re-ranking and explanation	Candidate catalogue, base score, allowed codes, schema validation, fallback
Ask Vidhya	Grounded multilingual explanation	Local retrieval, citations, service boundaries, fallback summary
Draft generation	Optional structured rewriting	Reviewed templates, preview, explicit finalization, no submission

Plan-task action	Model may propose one function call	Owner-scoped action context, allowlist, stored proposal, expiry, confirmation, atomic claim
Reminders	No LLM needed	Consent, due time, cadence, target completion check, bounded worker

Grounding flow

1. Tokenize the question and retrieve up to five entries from reviewed local catalogues.
2. Build citations from server-owned source metadata.
3. Supply only that trusted context plus at most ten recent messages.
4. Instruct the model to avoid eligibility, diagnosis, emergency priority, credit approval, fabricated fees/deadlines/ status, and sensitive-data collection.
5. Return the answer with citations and a mode label.
6. If the model call fails, return a deterministic grounded summary from the same sources.

Why structured output matters

Free-form model text is not a safe input to application state. For recommendation re-ranking, the API requests a strict JSON schema and then verifies every returned service code against the application-owned candidate map. For tool calls, the API accepts only the `propose_plan_task_update` function with enumerated plan types and exact plan/task identifiers.

Structured output reduces syntax ambiguity; it does not prove semantic correctness. The server must still validate ownership, existence, current state, allowed transitions, and expiry.

Why the agent is approval-gated

The model can propose only one reversible, low-impact action: mark an education or finance checklist task completed/ not-started.

- The proposal is matched against a compact list of the authenticated user's active plans.
- A proposal creates a pending action; it does not mutate the plan.
- The action expires after 15 minutes.
- The UI sends a second authenticated confirmation request.
- `findOneAndUpdate` atomically moves pending -> executing, so repeated confirmations cannot claim the same action twice.
- Execution rechecks the owner, active plan, and task.
- The final status is confirmed or failed, with an audit result.

This is "bounded agency," not general autonomy. Money movement, identity submission, document sharing, appointment booking, official application submission, and emergency priority are deliberately outside the action catalogue.

Agentic reminders are a workflow agent, not an LLM agent

The reminder worker scans active due records, checks whether the owned target is still incomplete, creates an in-app notification, advances the next run, or completes itself. The resident explicitly selects the target, first time, and cadence. It never infers an official deadline or changes the target record.

Current limitation: the worker runs inside the API process. With multiple instances it may duplicate work, and serverless runtimes are not reliable schedulers. The production redesign is a durable queue/scheduler with an atomic lease/idempotency key, retry policy, dead-letter handling, and observable worker metrics.

AI evaluation answer

Current evidence covers deterministic ranking, output parsing, fallback behavior, owner-scoped action validation, expiry, and confirmation in automated tests. A stronger production evaluation suite would add:

- A versioned golden set of resident intents in supported languages.
- Retrieval recall@k for the correct reviewed service.
- Groundedness/claim verification against supplied sources.
- Refusal tests for eligibility, diagnosis, payment, OTP, and emergency-priority requests.
- Tool-call precision and false-action rate.
- Fallback success under timeouts, malformed output, rate limits, and missing keys.
- Latency and cost percentiles by language and workflow.
- Human review for clarity, accessibility, and harm severity.

Do not claim the project has a mature AI evaluation platform if it does not. Say exactly what is tested and what you would add.

Model and cost discussion

The bounded tasks use low reasoning effort because the model is explaining reviewed data rather than solving an open research problem. Cost controls include small candidate sets, at most ten history messages, no model call when the key is absent, deterministic fallback, and no parallel tool calls. At scale, add token budgets, semantic cache for public guidance, per-user quotas, timeout/circuit breakers, and per-workflow cost dashboards.

8. Security and privacy deep dive

Why opaque server-side sessions instead of browser JWTs

- The browser stores only a random HTTP-only cookie, reducing token theft through JavaScript.
- MongoDB stores a SHA-256 token hash rather than the raw session token.
- Sessions support server-side revocation, device listing, an active-device limit, 30-minute idle expiry, seven-day absolute expiry, and TTL cleanup.
- The CSRF token stays in application memory and is required on authenticated mutations.
- Production uses a secure `__Host-` cookie with path `/` and no Domain attribute.

Trade-off: each authenticated request requires a session lookup, and availability depends on the session store. At higher scale, use an indexed/replicated session store, cache carefully, and preserve immediate revocation semantics.

CSRF reasoning

HTTP-only cookies are sent automatically by the browser, so they do not remove CSRF risk. Vidhya Vedha combines `SameSite=Lax`, an exact credentialed CORS allowlist, and a per-session CSRF token compared in constant time for non-GET/HEAD/OPTIONS requests. XSS remains a threat because injected JavaScript could read the CSRF token from application memory; CSP, dependency hygiene, safe rendering, and input/output handling remain required.

Google sign-in reasoning

The API issues a short-lived nonce in an HTTP-only cookie, sends the public client ID and nonce to the browser, verifies the returned Google ID token server-side against the configured audience, requires a verified email and matching nonce, and then creates the same local session used by password login. Account linking is intentionally conservative for non-Google-authoritative email domains.

Authorization model

Authentication proves who the caller is. Authorization still requires every owner query to include `userId`, and provider/admin routes use role middleware. Public identifiers such as `planId` or `confirmationCode` are not authorization secrets. Negative cross-tenant tests should accompany every new owner-scoped route.

Validation and injection controls

Zod schemas restrict accepted fields, types, lengths, enums, and identifiers. Payload sanitization rejects dangerous key patterns, services use explicit fields, and Mongoose queries use trusted operators where necessary. The design does not pass arbitrary client filters/operators into MongoDB.

Document vault

- PDF/JPEG/PNG only, maximum 5 MB.
- Bytes are held in memory only until encryption.
- AES-256-GCM uses a random 12-byte IV and authentication tag.
- The original filename is encrypted separately.
- MongoDB stores metadata and encryption parameters, while the local store contains ciphertext only.
- Reads require the owner and active consent; revocation destroys ciphertext and encrypted filename material.
- A missing/invalid dedicated key returns 503; it never falls back to the session secret.

Production limitation: local filesystem storage is unsuitable for horizontally scaled/serverless production. The next design uses managed object storage, envelope encryption with KMS/versioned data keys, malware scanning, key rotation, backup/restore tests, and auditable access logs.

Privacy-safe analytics

Blocker analytics accepts only a fixed taxonomy, not free text or service payloads. A keyed HMAC pseudonym deduplicates one user/service/stage/reason per day, raw events expire after 180 days, and admin results suppress groups smaller than three. Explain that pseudonymization is not anonymization; key governance and access controls still matter.

9. Data correctness, consistency, and failure handling

Important data patterns

Pattern	Where used	Why it matters
Owner compound queries	Plans, appointments, conversations, reminders, trackers, vault metadata	Prevents insecure direct object reference across residents.
Partial unique indexes	Active healthcare and home bookings	Preserves exclusivity while allowing cancelled history.
TTL indexes	Sessions, AI conversations/messages, agent actions, blocker events	Enforces retention and limits stale sensitive state.
Optimistic concurrency	Checklists, plans, cases, handoffs	Detects conflicting document updates.
Atomic state claim	Agent action pending -> executing	Makes repeated confirmation idempotent at the claim boundary.
Enums/state machines	Emergency, handoff, booking, reminder and draft statuses	Prevents invalid arbitrary transitions.
Redacted projections	Provider operations	Avoids leaking resident details into operational dashboards.

Failure semantics to mention

- 400/422: malformed or semantically invalid input.
- 401: no valid session.
- 403: valid identity without permission or invalid CSRF.
- 404: do not reveal another user's resource; an owner-scoped miss stays a miss.
- 409: valid request conflicts with current state, such as a booked slot.
- 429: rate limit.
- 503: required capability is unconfigured/unavailable, such as MongoDB or vault key.
- AI failure: return a deterministic grounded mode instead of a generic server failure.
- External connector absent: return an explicit not-connected capability, never a fabricated status.

Status connector design

Internal connectors read owned Vidhya healthcare, roadside, and home records. External providers remain not-connected until a reviewed adapter is registered. Registering an external adapter requires a provider entry marked official. The next production design should add connector-specific credentials, timeouts, retries with jitter, circuit breakers, signed webhook verification, normalized statuses, source timestamps, audit logs, and clear stale-data UX.

10. Delivery and production evidence

What is currently verifiable

- React/Vite frontend and Express API deploy together on Vercel under one HTTPS origin.
- MongoDB Atlas persists sessions and workflow records.

- /api/health/live separates process liveness from /api/health/ready database readiness.
- Pull requests run repository hygiene, frontend tests/build, backend tests, and dependency security policy.
- Vercel creates a preview deployment.
- A workflow after main CI and once daily runs 13 production checks: liveness, database readiness, nine service catalogues, Google configuration, and session protection.
- Main requires backend, frontend, dependency, hygiene, and Vercel checks.
- The verified baseline contains 61 frontend tests and 122 backend tests.

Test strategy explanation

Level	Examples	What it proves
Unit/domain	Slot generation, date rules, recommendation scoring, redaction, encryption, action parsing	Business logic across edge cases without HTTP.
API/middleware	Auth/session, CSRF, validation, ownership, roles, routes, error semantics	Security and HTTP contracts.
Concurrency/data	Duplicate booking indexes, collision responses, atomic action claim	Invariants under competing requests.
Frontend component	Journey rendering, form states, auth-aware behavior, API response handling	User-visible behavior and regressions.
Build/static	Vite production build and repository hygiene	Bundling and release cleanliness.
Production smoke	Health/readiness, catalogues, Google config, protected session endpoint	The deployed main release and environment actually work.

What production smoke does not prove

A smoke test is not a load test, backup restore, security assessment, browser matrix, accessibility audit, external-provider contract test, or full write-path synthetic transaction. Say this before the interviewer asks.

Current production gaps and the correct redesign

Current constraint	Risk	Next design
Process-local rate limiter	Limits are inconsistent across serverless instances	Redis/managed shared store, trusted client keying, abuse metrics
Reminder worker inside API	Duplicate/missed work across replicas/serverless pauses	Durable scheduler/queue, leases, idempotency, retry/DLQ
Local encrypted vault	Files do not survive/scale safely in serverless	Object storage, KMS envelope encryption, malware scan, lifecycle policy
Seeded provider data	No verified supply, tenancy, leave or SLA	Provider onboarding, contracts, tenant binding, schedule admin, audit trail
Limited centralized telemetry	Slow incident detection and diagnosis	Structured redacted logs, traces, metrics, alerts, SLOs, correlation IDs
No backup-restore evidence	Data recovery is assumed, not proven	Automated encrypted backups and scheduled restore drills

Official APIs not connected	No official submission/status	Reviewed adapters, sandbox tests, consent, credentials, monitoring, legal agreements
Single-region/simple topology	Regional outage and latency risk	Multi-region frontend, deliberate data residency/replication and failover plan

Observability answer

For a production service I would define SLIs and SLOs before adding dashboards:

- Availability and p95/p99 latency by API/workflow.
- Login success/failure and suspicious authentication rates.
- Booking conflict rate versus genuine error rate.
- MongoDB connection/error/slow-query metrics.
- AI latency, fallback rate, token cost, groundedness/refusal failures.
- Reminder due-to-delivery lag, retry count, and dead-letter size.
- Connector success, timeout, circuit state, and stale-status age.
- Vault encryption/read/delete failures without logging filenames or content.
- Client error funnels and privacy-safe blocker stages.

Every request should carry a correlation ID; logs should use a redaction policy and never include cookies, tokens, OTPs, document content, full contact data, or model secrets.

Scale-to-10-million answer

Do not say "MongoDB scales automatically." Use a staged design:

1. Split static/catalogue reads from authenticated workflow writes and cache versioned public catalogues at the CDN.
2. Move sessions and rate limits to managed low-latency shared stores while retaining revocation.
3. Introduce a durable event bus/queue for reminders, notifications, connector refresh, and document scanning.
4. Partition high-volume records by tenant/region or hashed owner key based on measured access patterns.
5. Build provider tenancy and authorization into the data model before onboarding organizations.
6. Put encrypted documents in regional object storage with KMS and asynchronous malware scanning.
7. Isolate unreliable external connectors behind worker pools, circuit breakers, and provider-specific budgets.
8. Add read replicas/materialized operational views where queues and analytics need different read patterns.
9. Define data residency, retention, deletion, disaster recovery, and regional failover objectives.
10. Load-test the dominant workflows and scale from measured bottlenecks, not guesses.

11. Core interviewer questions and strong answer points

Q1. What exact problem are you solving?

Strong answer: The gap is not access to links. It is service selection, preparation, trust boundaries, progress recovery, and assisted handoff for residents who may not know the responsible authority. The system reduces decision and readiness friction while keeping official actions with the authority.

Follow-up: What metric proves this? Measure task-selection success, readiness completion, official handoff click-through, time to next action, abandonment stage, assistance escalation, and incorrect-route rate. Do not claim social impact without user evidence.

Q2. Why is this not just a directory?

Strong answer: A directory stops at discovery. Vidhya Vedha maintains owner-scoped assessments, checklists, drafts, reminders, internal bookings/dispatch, capability-aware status, and human handoff. The important value is workflow continuity and safe boundaries, not the number of links.

Q3. What did you personally own?

Strong answer template: "I owned the architecture and implementation of X, Y, and Z. I made decision A because of constraint B, wrote tests C, and handled production issue D. I used external libraries/services for E, and the following parts remain fixtures." Replace placeholders with only work you can defend line by line.

Trap: saying "we built" when you cannot identify your own decisions, code, tests, and incidents.

Q4. Why did you choose a modular monolith?

Strong answer: It minimized distributed-system overhead for the current team/traffic while preserving domain boundaries in services/models. Extract only when a domain has independent scale, reliability, compliance, or ownership needs - likely reminders/connectors/vault first.

Q5. Walk me through one authenticated mutation.

Strong answer: Cookie session -> hash lookup and expiry/revocation check -> CSRF token -> rate limit -> Zod validation -> authentication/role -> owner-scoped query -> domain state check -> atomic/database write -> mapped response -> UI refresh.

Q6. Why not JWT in localStorage?

Strong answer: Opaque HTTP-only cookies reduce script-readable bearer exposure and give immediate server-side revocation, device management, idle expiry, and session limits. The trade-off is a database lookup and CSRF protection requirement.

Q7. If the cookie is HTTP-only, why do you need CSRF?

Strong answer: The browser still attaches cookies automatically to cross-site requests. SameSite and CORS help, but a per-session CSRF token on mutations creates an explicit proof that the request originated from the application context.

Q8. How do you prevent horizontal privilege escalation?

Strong answer: Never query a resident resource only by its public ID. Include authenticated userId in every read/update/delete filter, recheck it during action execution, restrict provider/admin routes by role, and add negative cross-tenant tests.

Q9. How do you stop double booking?

Strong answer: The API validates the candidate slot, but the final invariant is enforced by a partial unique MongoDB index on active provider/time. Competing writes serialize at the index; the loser receives duplicate-key and the API maps that to 409 SLOT_UNAVAILABLE.

Q10. Why a partial unique index?

Strong answer: Appointment history must remain after cancellation. The index applies only when status is booked, so a cancelled record no longer blocks the time while the audit/history record remains.

Q11. Could rescheduling still race?

Strong answer: Yes; the destination slot can be taken after availability is shown. The update must rely on the same unique constraint and handle conflict. For more complex multi-record moves, use a transaction or a reservation/lease design and define rollback semantics.

Q12. What does optimistic concurrency protect?

Strong answer: It detects lost updates when two clients edit the same document version. It is not a replacement for unique indexes or atomic state transitions, which protect different invariants.

Q13. How does Google login avoid replay/confusion attacks?

Strong answer: Issue a short-lived nonce in an HTTP-only cookie, pass it to Google Identity Services, verify the ID token server-side for audience, verified email, subject, and exact nonce, clear the nonce cookie, then create the same local opaque session.

Q14. How are passwords and sessions stored?

Strong answer: Passwords use bcrypt. The random session cookie is not stored raw; MongoDB stores a SHA-256 token hash. Session metadata has idle/absolute expiry, revocation and TTL cleanup. A dedicated server secret HMACs the observed IP for privacy-preserving session context.

Q15. How do rate limits work on Vercel?

Strong answer: Express trusts the Vercel proxy hop so the limiter sees the forwarded client address. Current storage is process-local, which is only per-instance protection. Production scale needs a shared store and alerting; never claim the current limiter is globally consistent.

Q16. What happens when MongoDB is down?

Strong answer: Database-backed routes return 503; public reviewed catalogues can remain available in degraded mode. Liveness can stay 200 while readiness becomes 503, allowing the platform to distinguish a running process from a usable dependency path.

Q17. How is the AI grounded?

Strong answer: Retrieve reviewed local entries, send only that context, require citations and safety instructions, validate structured identifiers against server-owned candidates, bound conversation history, and fall back deterministically on any model failure.

Q18. Can prompt injection make it submit an application?

Strong answer: No such tool exists. The model cannot gain capabilities through text. The only tool is a plan-task proposal; it is validated against owner-scoped context, stored pending, expires, and requires a second confirmation before an atomic claim and revalidation.

Q19. Why use AI if deterministic fallback works?

Strong answer: AI improves natural-language matching, multilingual explanation, and conversation, while deterministic logic preserves availability and safety. The system uses AI only where probabilistic behavior adds user value.

Q20. How do you evaluate hallucination?

Strong answer: Current tests verify fallback/schema/action boundaries. A production eval uses a golden multilingual set, retrieval recall, grounded claim scoring, sensitive-request refusal, tool-call precision, human severity review, and continuous monitoring of fallback/correction rates.

Q21. What makes the reminder feature agentic?

Strong answer: It observes due state, checks the target, decides whether to notify/advance/complete, and acts within prior consent. It is bounded workflow autonomy, not open-ended planning, and does not need an LLM.

Q22. How do you make reminders reliable at scale?

Strong answer: Move from an in-process timer to a durable scheduler/queue; atomically lease due jobs; use idempotency keys, retries with backoff, dead-letter queues, per-channel delivery state, and lag/failure metrics.

Q23. How is document encryption implemented?

Strong answer: AES-256-GCM with a random IV and authentication tag; filenames are encrypted separately; only ciphertext is written; metadata and ownership live in MongoDB; access requires owner and active consent. The current local store is a prototype, not a serverless production store.

Q24. Why is AES-GCM preferable here?

Strong answer: It provides confidentiality and integrity/authenticity. Decryption fails if ciphertext, IV, tag, or key is wrong. Production still needs KMS, envelope-key versioning, rotation, malware scanning, storage IAM, and backup design.

Q25. Are blocker analytics anonymous?

Strong answer: They are pseudonymized, not fully anonymous. The fixed taxonomy and HMAC reduce exposure, TTL limits retention, and small-group suppression reduces re-identification, but the key and access still require governance.

Q26. Do all status checks call official government APIs?

Strong answer: No. Internal healthcare, roadside, and home-service workflows have working status data. External government providers remain explicitly "not_connected" until a registered adapter and credentials exist. The UI never presents fixture or inferred status as an official response.

Q27. How would you integrate an official status API safely?

Strong answer: Put it behind a provider adapter with a normalized contract, strict timeouts, circuit breaking, retry rules that respect idempotency, response-schema validation, audit logging, and a visible source/timestamp. Keep secrets server-side and degrade to a clear unavailable state.

Q28. How do provider dashboards protect resident data?

Strong answer: Role and organization scope are checked server-side. List views expose only operational fields and redact names, phone numbers, reasons, and precise location. Detailed access should be purpose-bound, audited, short-lived, and tested with cross-tenant negative cases.

Q29. What does CI prove?

Strong answer: It proves repository hygiene, dependency policy, backend tests, frontend tests, and a production build for the exact commit. It raises confidence; it does not prove runtime integrations, accessibility quality, disaster recovery, or production health.

Q30. Why run production smoke tests too?

Strong answer: CI validates an artifact in an isolated environment. Smoke tests validate the deployed routing, security headers, catalogues, authentication boundaries, and critical public paths after release and on a schedule. They are intentionally small and non-destructive.

Q31. What is the deployment and rollback strategy?

Strong answer: Protected main requires checks and review, Vercel creates immutable deployments, and post-deploy smoke tests gate confidence. For a bad release, promote the previous known-good deployment or revert the merge; database changes must follow backward-compatible expand/migrate/contract steps.

Q32. What would you monitor first?

Strong answer: Availability and latency by route, 5xx/error codes, Mongo connection health, booking conflicts, queue lag, notification failures, AI fallback/schema-rejection rates, connector timeouts, authorization denials, and user funnel drop-off. Every request should carry a correlation ID.

Q33. What is your backup and disaster-recovery plan?

Strong answer: Define RPO and RTO first. Use Atlas backups and restore drills, separately version and back up encrypted objects, protect encryption keys in KMS, document region/account recovery, and regularly test restoration into an isolated environment. A backup that has not been restored is only an assumption.

Q34. How would this handle ten million residents?

Strong answer: Keep stateless API instances, shared rate limits, cache immutable catalogues, index owner/status/due queries, paginate every list, move reminders to queues, place documents in object storage, isolate analytics writes, and add capacity tests. Split services only where load or ownership justifies it.

Q35. What would you extract first from the monolith?

Strong answer: The reminder/notification worker, because it has independent scaling, retry, and scheduling behavior. Next could be document processing because its storage, scanning, and security boundary differ. I would not split CRUD domains merely to claim microservices.

Q36. Why MongoDB rather than a relational database?

Strong answer: The document model fits evolving service catalogues, plans, and workflow metadata, and MongoDB supports the required unique and TTL indexes. Booking and workflow invariants still need careful index/state design. PostgreSQL would also be valid, especially if cross-entity transactions and reporting dominate.

Q37. What consistency model does the user see?

Strong answer: User-owned mutations require read-your-write behavior from the primary path. Unique indexes and atomic state transitions protect critical invariants. Analytics, reminders, and external status refresh can be eventually consistent if the UI displays timestamps and pending states.

Q38. How do you design for rural users?

Strong answer: Mobile-first pages, low payloads, plain-language steps, explicit official handoffs, save-and-resume plans, deterministic service availability during AI failure, local-language support, and assisted human handoff. The next validation step is field research on device, network, literacy, and trust constraints.

Q39. What would multilingual voice add, and what new risks appear?

Strong answer: It can reduce literacy and keyboard barriers. Risks include transcription errors, language-model drift, background speech capture, sensitive audio retention, and inaccessible correction. Show transcripts for confirmation, minimize retention, disclose processing, and never let voice bypass approvals.

Q40. How do you measure product impact?

Strong answer: Measure successful service selection, checklist completion, draft acceptance after review, booking completion, assisted-handoff resolution, time-to-completion, and repeated blocker categories. Pair funnel metrics with user interviews; clicks alone do not prove a resident received a service.

Q41. What is the biggest engineering compromise?

Strong answer: Some production boundaries are demonstrated rather than fully externalized: local encrypted file storage, in-process reminders, process-local rate limits, and unavailable official adapters. I label those honestly and can explain the production replacement for each.

Q42. What would you change with a five-person team?

Strong answer: Assign clear ownership to identity/security, resident workflows, provider operations, platform/reliability, and product/frontend. First harden three workflows, create contracts and SLOs, add external stores/queues/KMS, run field research, and integrate one real provider end to end.

Q43. What are the primary cost drivers?

Strong answer: Database/storage, document object storage and scanning, notification delivery, observability, and AI tokens. Bound AI context and output, cache reviewed content, prefer deterministic matching when sufficient, tier storage, and measure cost per completed workflow rather than cost per request.

Q44. Describe a production incident you would rehearse.

Strong answer: A booking endpoint starts returning elevated 500s after deployment. Freeze rollout, compare error rate by version, inspect correlation IDs and Mongo errors, promote the last good deployment, verify smoke/SLO recovery, preserve evidence, then write a blameless postmortem with a test or guardrail preventing recurrence.

Q45. What would you build in the next 90 days?

Strong answer: Stabilize the three flagship workflows; replace local files, timers, and local limits with object storage/KMS, queues, and a shared limiter; add observability and restore drills; integrate one verified provider; create AI evals; and validate accessibility with real residents and service-centre operators.

12. System-design drills

Drill A: high-scale healthcare booking

Requirements: browse slots, book/cancel/reschedule, no double booking, auditable provider changes, and graceful hot-slot contention.

Design:

1. Treat provider, start time, duration, status, and tenant as the invariant-bearing record.
2. Generate availability from schedules but confirm only through a database constraint.
3. Use an idempotency key so retries do not duplicate bookings.
4. Keep a partial unique index for active provider/time and translate conflict to 409.
5. Publish an outbox event after commit for reminders and provider updates.
6. Cache schedule reads, never booking writes; paginate history.
7. Track conflict rate, booking latency, stale-slot rate, and notification lag.

Discuss: recurring schedules, time zones, multi-slot appointments, waitlists, cancellation policy, and what must be transactional.

Drill B: durable reminders

Requirements: millions of due tasks, resident consent, retries, multiple channels, no duplicate spam.

Design:

1. Persist reminder intent and next-run time with owner, channel, consent version, and state.
2. Poll indexed due rows or use a scheduler to enqueue jobs.
3. Atomically lease work; attach an idempotency key per task/channel/window.
4. Recheck consent and current task state immediately before delivery.
5. Retry transient errors with jittered backoff; dead-letter permanent failures.
6. Store provider receipt/failure without storing unnecessary message content.
7. Monitor queue lag, success rate, duplicate suppression, opt-outs, and cost.

Drill C: official status connector platform

Requirements: providers have different authentication, schemas, limits, latency, and outage behavior.

Design:

1. Define one internal status contract: state, message, source, observed time, provider reference, and confidence.

2. Register allowlisted adapters with per-provider credentials and schema validators.
3. Apply timeout, circuit breaker, rate limit, retry, and cache policy per adapter.
4. Separate provider errors from "not_connected", "not_found", and user input errors.
5. Show source and freshness in the UI; never silently substitute fixture data.
6. Audit access without logging credentials or sensitive payloads.
7. Use contract tests and sandbox fixtures supplied by the provider.

Drill D: encrypted document vault

Requirements: consent-controlled uploads, serverless deployment, rotation, deletion, and auditability.

Design:

1. Upload through authenticated, size/type-limited endpoints; malware scan before trusted use.
2. Generate a per-object data key; encrypt using authenticated encryption.
3. Wrap the data key with KMS and store key version plus IV/tag metadata.
4. Store ciphertext in private object storage and metadata in MongoDB.
5. Authorize every access by owner, purpose, consent, and short-lived download capability.
6. Record immutable audit events; keep filenames and logs privacy-minimized.
7. Rotate wrapping keys, define cryptographic deletion, and test restore/rotation procedures.

13. Live demo script (eight minutes)

0:00-0:45 - frame the problem

Say the one-sentence pitch, name the resident/provider roles, and state that the product completes preparation and operational workflows while preserving explicit official handoffs.

0:45-2:15 - resident companion

1. Sign in with a demo account.
2. Enter a need in the eligibility/service guide.
3. Show the reviewed recommendation and why it matched.
4. Create a plan, open its personalized checklist, and show save/resume state.
5. Generate a draft and emphasize preview before PDF download.

Narrate the boundary: AI may explain and rank; the server validates IDs and the resident confirms mutations.

2:15-3:45 - healthcare correctness

1. Browse a provider and time slot.
2. Book it and show status/history.
3. Explain the partial unique index and 409 conflict behavior.
4. Mention owner scoping and the modification cutoff.

3:45-5:00 - roadside operations

1. Create a request with consented location.
2. Show provider queue/status movement.
3. Explain what is redacted on list views and how tenant/role checks work.

5:00-6:15 - AI safety and agent action

Show a grounded assistant answer with citations, then an action proposal. Stop before confirmation and explain expiration, atomic claim, owner recheck, and the fact that no arbitrary application-submission tool exists.

6:15-7:15 - engineering proof

Open the architecture diagram, CI workflow, tests, production smoke checks, and one representative security test. State the exact current test counts rather than saying "fully tested."

7:15-8:00 - honest close

Name the four production substitutions still needed and the next 90-day plan. Invite the interviewer to choose a deep dive: security, booking consistency, AI guardrails, or reliability.

Demo failure fallback

Keep a short screen recording or screenshots, a known demo account, seeded non-sensitive data, the deployed health endpoint, and the relevant code/index definitions ready. Do not debug silently; explain the failing layer and continue with architecture evidence.

14. Questions likely to interrupt the demo

Interruption	Best response pattern
"Is that real data?"	Identify reviewed catalogue, persisted user data, fixture/demo data, and external provider data separately.
"What if OpenAI is unavailable?"	Show deterministic fallback and describe the user-visible degraded state.
"Can two people book this?"	Point to the database constraint, not only the UI check.
"Can I change another user's ID?"	Explain owner-scoped query filters and show the negative authorization test.
"Does this submit to government?"	Say no unless an official adapter exists; show the handoff notice.
"Why should I trust the recommendation?"	Show reviewed candidates, explanation, citations, and final user control.
"Where are files stored in production?"	State that local disk is a prototype and present the object-store/ KMS design.
"What happens after a serverless restart?"	Persisted workflows remain; the in-process reminder loop is the gap replaced by a durable queue.

15. Questions to ask the interviewer

Ask two or three, based on the conversation:

1. Which failure modes or security boundaries would you want the team to design first for a product handling resident documents?
2. How does your organization decide when an AI feature has enough evaluation evidence to ship?
3. Where do you draw service boundaries: team ownership, scale, compliance, or deployment cadence?
4. What production signals distinguish a successful release from a technically healthy release?
5. How are engineers expected to participate in threat modeling and incident response?

These questions show engineering judgment. Avoid asking something already answered in the job description.

16. Facts to memorize

Fact	Interview-ready value
Frontend	React + Vite SPA
Backend	Express API deployed with the application
Database	MongoDB Atlas
Authentication	Password plus optional Google Identity Services; opaque server sessions
Session policy	30-minute idle, 7-day absolute, TTL cleanup, device/session controls
Booking invariant	Partial unique index on active provider/time; conflict returns 409
AI strategy	Reviewed-candidate grounding, structured allowlisted output, deterministic fallback
Agent action	One allowlisted plan-task mutation, pending proposal, 15-minute expiry, second confirmation
Vault prototype	AES-256-GCM; production replacement is object storage plus KMS and scanning
Analytics	Fixed taxonomy, HMAC pseudonym, 180-day TTL, suppress groups below three
Automated tests	61 frontend and 122 backend tests at guide publication
Deployment proof	Protected main, CI, Vercel deployment, post-deploy and scheduled smoke checks

Recheck counts and operational facts before every interview. Never memorize claims that the repository no longer proves.

17. Resume-ready bullets

Use only after tailoring to the job and verifying metrics:

- Built and deployed a full-stack digital-service companion spanning guided discovery, document readiness, appointment booking, dispatch, and provider operations using React, Express, and MongoDB.
- Designed secure opaque-cookie sessions, per-session CSRF protection, Google ID-token verification, owner-scoped authorization, validation, rate limiting, and privacy-minimized analytics.
- Implemented grounded AI service guidance with reviewed candidates, schema-validated outputs, deterministic fallback, and an approval-gated agent action rather than unrestricted autonomous tools.
- Protected concurrent appointment booking with database-level partial unique indexes and explicit conflict semantics; added 183 automated frontend/backend tests plus deployment smoke checks.
- Documented production gaps and designed migrations to durable queues, shared rate limits, object storage/KMS, provider adapters, observability, and disaster recovery.

Do not claim user scale, revenue, latency improvement, model accuracy, or official integrations without measured evidence.

18. Self-scoring rubric

Score each dimension from 1 to 5 after a mock interview.

Dimension	1	3	5
Problem framing	Feature list	User and problem clear	Impact, boundary, and evidence are crisp
Architecture	Names tools	Explains request flow	Defends trade-offs and evolution
Correctness	Relies on validation	Names indexes/state	Explains races, idempotency, failure semantics
Security/privacy	Says "secure"	Names controls	Threat model, residual risks, operational controls
AI judgment	Calls an API	Grounding/fallback	Evals, tool boundaries, cost, failure handling
Reliability	Says CI exists	Tests and health checks	SLOs, monitoring, rollback, DR, capacity
Ownership	Says "we"	Names contributions	Explains decisions, mistakes, and measurable outcome
Communication	Long/unclear	Structured answer	Concise claim, evidence, trade-off, next step

Target: no score below 4 for the three areas most important to the role.

19. Final rehearsal checklist

- Can explain the product in one sentence, 90 seconds, five minutes, and eight minutes.
- Can draw the architecture from memory and follow one mutation end to end.
- Can explain one database invariant and one race condition precisely.
- Can explain authentication, CSRF, authorization, encryption, and privacy separately.
- Can distinguish deterministic logic, generative AI, and agentic workflow behavior.
- Can describe AI evaluation without claiming that unit tests measure model quality.
- Can distinguish internal working services, demo data, and unavailable official connectors.
- Can name current production gaps and the concrete replacement for each.
- Can describe monitoring, rollback, backup/restore, and a rehearsed incident.
- Can show CI, tests, deployment, and live health evidence.
- Can say exactly what you personally designed, implemented, tested, and changed.
- Can answer "why this design?" and "what would you change?" without defensiveness.

20. Answer framework when you get stuck

Use: Requirement -> invariant -> design -> failure -> evidence -> next step.

Example: "The requirement is that one active provider slot cannot be sold twice. The invariant lives in a partial unique index. The API validates availability and converts duplicate-key to 409. A stale UI can still race, but the database remains correct. A concurrency test proves the behavior. At higher scale I would add idempotency and measure conflict rate."

If you do not know, say what you know, identify the missing assumption, propose how you would verify it, and avoid inventing a production fact.

21. Closing statement

Vidhya Vedha is strongest in an interview when presented as an engineering case study in safe workflow completion: honest integration boundaries, database-enforced correctness, layered security, grounded AI with narrow tools, and a credible production-hardening plan. The goal is not to claim that every public service has been digitized. The goal is to prove that you can reason about a consequential product from user need through failure handling, deployment, and evolution.